

1. Feedback

Tema a fost interesanta si utila, deoarece a implicat procesarea unui volum mare de date JSON si aplicarea conceptelor de paralelizare pe un pipeline complet (citire - procesare - analiza - scriere).

Mi-a placut faptul ca am avut libertatea de a alege strategia de paralelizare, ceea ce m-a fortat sa gandesc arhitectura unei solutii scalabile si thread-safe.

Provocarile principale au fost: parsarea JSON-ului, asigurarea thread safety, obtinerea unei scalabilitati bune atunci cand numarul de thread-uri creste.

2. Strategia de paralelizare

Solutia este impartita in doua faze majore care ruleaza in paralel, urmata de etape secventiale necesare pentru output.

A) Citirea fisierelor JSON (parallel I/O)

In metoda `ParallelProcessor.loadArticles`, folosesc un pool de thread-uri de dimensiune adaptiva (`min(numThreads, nrFisiere)`).

Fiecare thread proceseaza un fisier independent:

- citește conținutul JSON,
- parsează articolele cu `SimpleJsonParser`,
- returnează lista de articole.

Aceasta etapa este eficienta pentru ca: I/O-ul se soldeaza bine in paralel, fisierele sunt independente, nu exista resurse partajate care sa creeze blocaje

B) Procesarea articolelor (parallel CPU-bound)

Acesta este etapa principala a solutiei si cea in care se obtine acceleratia reala.

1) Impartirea in chunk-uri

Lista de articole este impartita in bucati fixe (`chunk_size = 800`). Fiecare chunk este procesat de un task separat in `Exercutro Service`.

Avantaje: workload echilibrat, overhead minim, utilizarea eficienta a procesorului.

2) Procesarea fiecarui articol

Fiecare thread executa urmatoarele operatii:

- normalizarea limbii articolului,
- extragerea si validarea categoriilor,
- actualizarea statisticilor globale prin `StatisticsCollector` (thread-safe),
- extragerea de keywords pentru articolele in limba engleza,
- actualizarea contorului global de keywords prin `ConcurrentHashMap`.

Structurile folosite sunt thread-safe: ConcurrentHashMap, ConcurrentLinkedQueue, AtomicInteger, AtomicReference

C) Eliminarea duplicatelor

Am ales sa fac deduplicarea secvential, deoarece operatia este $O(n)$ si nu reprezinta un bottleneck comparativ cu procesarea paralela a articolelor.

D) Generarea fisierelor de output

Output-ul este generat secvential, iar etapele includ:

- sortarea articolelor dupa data pentru all_articles.txt
- generarea fisierelor pe limbi, categorii
- generarea fisierului keywords_count.txt
- raportul final reports.txt

3. Analiza de performanta si scalabilitate

Setup: 16GB RAM, Procesor 13th Gen Intel(R) Core(TM) i5-13450HX (2.40 GHz), Windows 11

Am rulat pentru input-ul din testul 4 dat in arhiva checker-ului si am repetat fiecare configuratie de regulare de 3 ori, secvential, cu 2 thread-uri si cu respectiv 4 thread-uri. La fiecare rulare secventiala am obtinut urmasori timpi:

Secvential: 1m23.477s – primul run, cand sunt citite fisierele de pe disc, 1m18.580s – al doilea run, 1m4.385s al treilea run

Rulare cu 2 thread-uri: 0m27.967s – primul run, 0m30.744s – al doilea run, 0m30.513s – al treilea run

Rulare cu 4 thread-uri: 0m20.773 – primul run, 0m23.014 – al doilea run, 0m21.919s – al trilea run

Medie secventiala: $(83.477s + 78.580 + 64.385) / 3 = 75.4807s$

Medie 2 thread-uri: $(27.967 + 30.744 + 30.513) / 3 = 29.74113s$

Medie 4 thread-uri: $(20.773 + 23.014 + 21.919) / 3 = 21.902s$

Acceleratie(speedup) : $S(p) = T1/Tp$

Speedup la 2 thread-uri: $S(2) = 75.48 / 29.74 = 2.54$

Seedup la 4 thread-uri: $S(4) = 75.48 / 21.90 = 3.45$

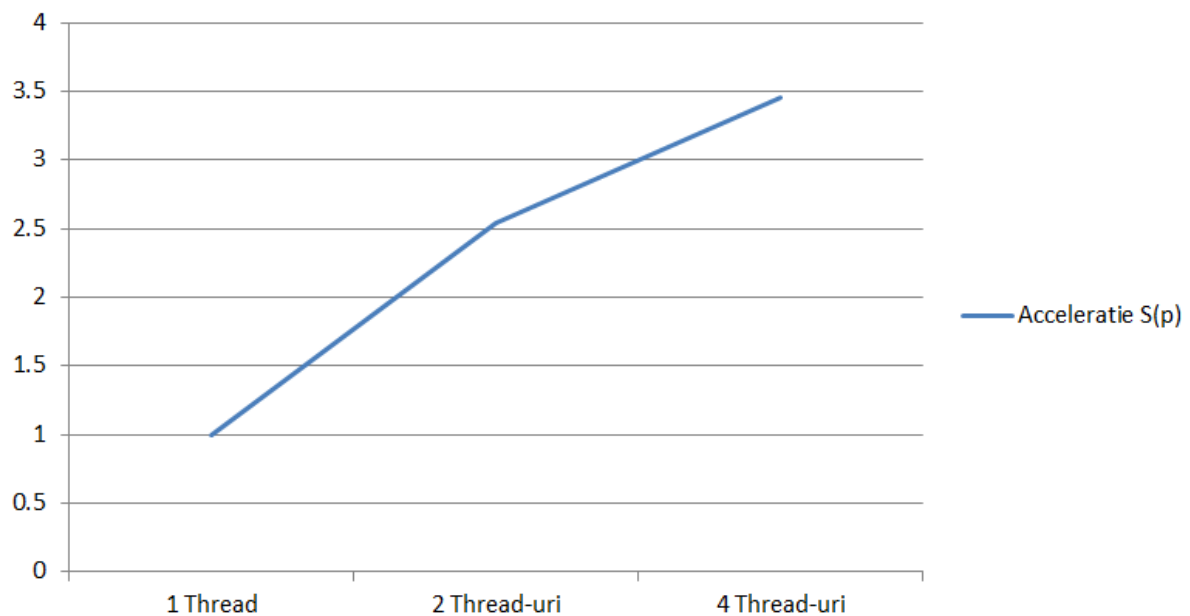
Eficienta: $E(p) = S(p) / p$

Eficient la 2 thread-uri: $E(2) = 2.54 / 2 = 1.27 (127\%)$

Eficienta la 4 thread-uri: $E(4) = 3.45 / 4 = 0.862 (86\%)$

Configuratie	Run 1	Run 2	Run 3	Medie	Acceleratie S(p)	Eficienta E(P)
1 Thread	83.477s	78.580s	64.385s	75.4807s	1	100%
2 Thread-uri	27.967s	30.744s	30.513s	29.74113s	2.54	127%
4 Thread-uri	20.773s	23.014s	21.919s	21.902s	3.45	86%

Acceleratie S(p)



Timp(s)

